

```

1  /* =====ONLY FOR UNDERSTANDING PURPOSE OF T3 CONCEPTS=====
2  Certainly! Here's a structured definition of the flight management system in an
   instructional format:
3
4  ### Flight Management System Definition
5
6  #### Classes and Data Structures
7
8  1. **Flight Class**
9     - **Attributes:** `flightNumber` (String), `origin` (String), `destination` (String)
10    - **Methods:**
11        - Constructor: Initializes `flightNumber`, `origin`, and `destination`.
12        - Getters and setters for `flightNumber`, `origin`, and `destination`.
13        - `toString()` method to return a string representation of the flight details.
14
15  2. **Customer Class**
16     - **Attributes:** `customerId` (String), `name` (String)
17     - **Methods:**
18        - Constructor: Initializes `customerId` and `name`.
19        - Getters and setters for `customerId` and `name`.
20        - `toString()` method to return a string representation of the customer details.
21
22  3. **Booking Class**
23     - **Attributes:** `bookingId` (String), `customer` (Customer), `flight` (Flight),
24       `seatNumber` (String)
25     - **Methods:**
26        - Constructor: Initializes `bookingId`, `customer`, `flight`, and `seatNumber`.
27        - Getters and setters for `bookingId`, `customer`, `flight`, and `seatNumber`.
28        - `toString()` method to return a string representation of the booking details.
29
30  4. **FlightManager Class**
31     - **Attributes:**
32        - `flights`: `HashMap` to store flights (`flightNumber` as key, `Flight` as value).
33        - `customers`: `HashSet` to store customers.
34        - `bookingsQueue`: `ArrayDeque` to manage bookings in FIFO order.
35        - `bookingsPriorityQueue`: `PriorityQueue` to manage bookings based on booking ID.
36     - **Methods:**
37        - `addFlight(String flightNumber, String origin, String destination)`: Adds a new
38          flight to the `flights` map.
39        - `getFlight(String flightNumber)`: Retrieves a flight from the `flights` map based
40          on `flightNumber`.
41        - `listFlights()`: Prints details of all flights stored in the `flights` map.
42        - `addCustomer(String customerId, String name)`: Adds a new customer to the
43          `customers` set.
44        - `getCustomer(String customerId)`: Retrieves a customer from the `customers` set
45          based on `customerId`.
46        - `listCustomers()`: Prints details of all customers stored in the `customers` set.
47        - `bookCustomer(String bookingId, Customer customer, Flight flight, String
48          seatNumber)`: Books a customer on a flight and adds the booking to both
49          `bookingsQueue` and `bookingsPriorityQueue`.
50        - `listBookings()`: Prints details of all bookings stored in the `bookingsQueue`.
51
52  5. **FileManager Class**
53     - **Attributes:** Constants for file paths (`FLIGHTS_FILE`, `CUSTOMERS_FILE`,
54       `BOOKINGS_FILE`).

```

```

47     - **Methods:**
48     - `saveFlights(Map<String, Flight> flights)`: Saves flights data to a text file
      (`FLIGHTS_FILE`).
49     - `loadFlights()`: Loads flights data from the text file (`FLIGHTS_FILE`) into a
      `HashMap`.
50     - `saveCustomers(Set<Customer> customers)`: Saves customers data to a text file
      (`CUSTOMERS_FILE`).
51     - `loadCustomers()`: Loads customers data from the text file (`CUSTOMERS_FILE`)
      into a `HashSet`.
52     - `saveBookings(Queue<Booking> bookings)`: Saves bookings data to a text file
      (`BOOKINGS_FILE`).
53     - `loadBookings()`: Loads bookings data from the text file (`BOOKINGS_FILE`) into a
      `PriorityQueue`.
54
55 #### Vacation Class (`Vacation.java`)
56
57 - **Vacation Class:**
58 - Acts as the entry point for the flight management system.
59 - Initializes a `FlightManager` instance.
60 - Loads initial data from files (`flights.txt`, `customers.txt`, `bookings.txt`) using
  methods from `FileManager`.
61 - Performs operations such as adding flights, customers, booking customers on flights,
  and saving data back to files using methods from `FlightManager`.
62 - Displays lists of flights, customers, and bookings using methods from
  `FlightManager`.
63
64 ### Explanation
65
66 - **Encapsulation:** Each class encapsulates its data (`Flight`, `Customer`, `Booking`)
  and provides methods to access and modify this data, ensuring data integrity and
  abstraction.
67 - **Data Structures:** Utilizes `HashMap`, `HashSet`, `ArrayDeque`, and `PriorityQueue`
  for efficient storage and management of flights, customers, and bookings.
68 - **File Handling:** `FileManager` class handles file operations (`load` and `save`)
  using `BufferedReader`, `BufferedWriter` for text-based file handling and
  `RandomAccessFile` for direct access operations.
69 - **Main Program Flow:** The `Vacation` class demonstrates typical operations of
  initializing the system, performing CRUD operations on flights, customers, and bookings,
  and persisting data to files.
70
71 */
72
73 import java.io.*;
74 import java.util.*;
75
76 // Flight class
77 class Flight {
78     String flightNumber;
79     String origin;
80     String destination;
81
82     public Flight(String flightNumber, String origin, String destination) {
83         this.flightNumber = flightNumber;
84         this.origin = origin;
85         this.destination = destination;

```

```

86     }
87
88     // Getters and setters
89     public String getFlightNumber() {
90         return flightNumber;
91     }
92
93     public void setFlightNumber(String flightNumber) {
94         this.flightNumber = flightNumber;
95     }
96
97     public String getOrigin() {
98         return origin;
99     }
100
101     public void setOrigin(String origin) {
102         this.origin = origin;
103     }
104
105     public String getDestination() {
106         return destination;
107     }
108
109     public void setDestination(String destination) {
110         this.destination = destination;
111     }
112
113     @Override
114     public String toString() {
115         return "Flight{" +
116             "flightNumber='" + flightNumber + '\'' +
117             ", origin='" + origin + '\'' +
118             ", destination='" + destination + '\'' +
119             '}';
120     }
121 }
122
123 // Customer class
124 class Customer {
125     String customerId;
126     String name;
127
128     public Customer(String customerId, String name) {
129         this.customerId = customerId;
130         this.name = name;
131     }
132
133     // Getters and setters
134     public String getCustomerId() {
135         return customerId;
136     }
137
138     public void setCustomerId(String customerId) {
139         this.customerId = customerId;
140     }

```

```
141
142     public String getName() {
143         return name;
144     }
145
146     public void setName(String name) {
147         this.name = name;
148     }
149
150     @Override
151     public String toString() {
152         return "Customer{" +
153             "customerId='" + customerId + '\'' +
154             ", name='" + name + '\'' +
155             '}';
156     }
157 }
158
159 // Booking class
160 class Booking {
161     String bookingId;
162     Customer customer;
163     Flight flight;
164     String seatNumber;
165
166     public Booking(String bookingId, Customer customer, Flight flight, String seatNumber)
167     {
168         this.bookingId = bookingId;
169         this.customer = customer;
170         this.flight = flight;
171         this.seatNumber = seatNumber;
172     }
173
174     // Getters and setters
175     public String getBookingId() {
176         return bookingId;
177     }
178
179     public void setBookingId(String bookingId) {
180         this.bookingId = bookingId;
181     }
182
183     public Customer getCustomer() {
184         return customer;
185     }
186
187     public void setCustomer(Customer customer) {
188         this.customer = customer;
189     }
190
191     public Flight getFlight() {
192         return flight;
193     }
194
195     public void setFlight(Flight flight) {
```

```

195         this.flight = flight;
196     }
197
198     public String getSeatNumber() {
199         return seatNumber;
200     }
201
202     public void setSeatNumber(String seatNumber) {
203         this.seatNumber = seatNumber;
204     }
205
206     @Override
207     public String toString() {
208         return "Booking{" +
209             "bookingId='" + bookingId + '\'' +
210             ", customer='" + customer +
211             ", flight='" + flight +
212             ", seatNumber='" + seatNumber + '\'' +
213             '}';
214     }
215 }
216
217 // FlightManager class
218 class FlightManager {
219     HashMap<String, Flight> flights = new HashMap<>();
220     HashSet<Customer> customers = new HashSet<>();
221     ArrayDeque<Booking> bookingsQueue = new ArrayDeque<>();
222     PriorityQueue<Booking> bookingsPriorityQueue = new PriorityQueue<>(Comparator.
        comparing(Booking::getBookingId));
223
224     // Method to add a flight
225     public void addFlight(String flightNumber, String origin, String destination) {
226         Flight flight = new Flight(flightNumber, origin, destination);
227         flights.put(flightNumber, flight);
228     }
229
230     // Method to retrieve a flight
231     public Flight getFlight(String flightNumber) {
232         return flights.get(flightNumber);
233     }
234
235     // Method to list all flights
236     public void listFlights() {
237         System.out.println("List of Flights:");
238         for (Flight flight : flights.values()) {
239             System.out.println(flight);
240         }
241     }
242
243     // Method to add a customer
244     public void addCustomer(String customerId, String name) {
245         Customer customer = new Customer(customerId, name);
246         customers.add(customer);
247     }
248

```

```

249 // Method to retrieve a customer
250 public Customer getCustomer(String customerId) {
251     for (Customer customer : customers) {
252         if (customer.getCustomerId().equals(customerId)) {
253             return customer;
254         }
255     }
256     return null; // Customer not found
257 }
258
259 // Method to list all customers
260 public void listCustomers() {
261     System.out.println("List of Customers:");
262     for (Customer customer : customers) {
263         System.out.println(customer);
264     }
265 }
266
267 // Method to book a customer on a flight
268 public void bookCustomer(String bookingId, Customer customer, Flight flight, String
seatNumber) {
269     Booking booking = new Booking(bookingId, customer, flight, seatNumber);
270     bookingsQueue.offer(booking);
271     bookingsPriorityQueue.offer(booking);
272 }
273
274 // Method to list all bookings
275 public void listBookings() {
276     System.out.println("List of Bookings:");
277     for (Booking booking : bookingsQueue) {
278         System.out.println(booking);
279     }
280 }
281 }
282
283 // FileManager class for file handling
284 class FileManager {
285     static final String FLIGHTS_FILE = "flights.txt";
286     static final String CUSTOMERS_FILE = "customers.txt";
287     static final String BOOKINGS_FILE = "bookings.txt";
288
289     // Method to save flights to file
290     public static void saveFlights(Map<String, Flight> flights) throws IOException {
291         BufferedWriter writer = new BufferedWriter(new FileWriter(FLIGHTS_FILE));
292         for (Flight flight : flights.values()) {
293             writer.write(flight.getFlightNumber() + "," + flight.getOrigin() + "," +
flight.getDestination());
294             writer.newLine();
295         }
296         writer.close();
297     }
298
299     // Method to load flights from file
300     public static HashMap<String, Flight> loadFlights() throws IOException {
301         HashMap<String, Flight> flights = new HashMap<>();

```

```

302     BufferedReader reader = new BufferedReader(new FileReader(FLIGHTS_FILE));
303     String line;
304     while ((line = reader.readLine()) != null) {
305         String[] parts = line.split(",");
306         String flightNumber = parts[0];
307         String origin = parts[1];
308         String destination = parts[2];
309         Flight flight = new Flight(flightNumber, origin, destination);
310         flights.put(flightNumber, flight);
311     }
312     reader.close();
313     return flights;
314 }
315
316 // Method to save customers to file
317 public static void saveCustomers(Set<Customer> customers) throws IOException {
318     BufferedWriter writer = new BufferedWriter(new FileWriter(CUSTOMERS_FILE));
319     for (Customer customer : customers) {
320         writer.write(customer.getCustomerId() + "," + customer.getName());
321         writer.newLine();
322     }
323     writer.close();
324 }
325
326 // Method to load customers from file
327 public static HashSet<Customer> loadCustomers() throws IOException {
328     HashSet<Customer> customers = new HashSet<>();
329     BufferedReader reader = new BufferedReader(new FileReader(CUSTOMERS_FILE));
330     String line;
331     while ((line = reader.readLine()) != null) {
332         String[] parts = line.split(",");
333         String customerId = parts[0];
334         String name = parts[1];
335         Customer customer = new Customer(customerId, name);
336         customers.add(customer);
337     }
338     reader.close();
339     return customers;
340 }
341
342 // Method to save bookings to file
343 public static void saveBookings(Queue<Booking> bookings) throws IOException {
344     BufferedWriter writer = new BufferedWriter(new FileWriter(BOOKINGS_FILE));
345     for (Booking booking : bookings) {
346         writer.write(booking.getBookingId() + "," +
347             booking.getCustomer().getCustomerId() + "," +
348             booking.getFlight().getFlightNumber() + "," +
349             booking.getSeatNumber());
350         writer.newLine();
351     }
352     writer.close();
353 }
354
355 // Method to load bookings from file
356 public static PriorityQueue<Booking> loadBookings() throws IOException {

```

```

357     PriorityQueue<Booking> bookings = new PriorityQueue<>(Comparator.comparing(
358         Booking::getBookingId));
359     BufferedReader reader = new BufferedReader(new FileReader(BOOKINGS_FILE));
360     String line;
361     while ((line = reader.readLine()) != null) {
362         String[] parts = line.split(",");
363         String bookingId = parts[0];
364         String customerId = parts[1];
365         String flightNumber = parts[2];
366         String seatNumber = parts[3];
367         Customer customer = new Customer(customerId, ""); // Dummy name, to be
            updated after loading customers
368         Flight flight = new Flight(flightNumber, "", ""); // Dummy
            origin/destination, to be updated after loading flights
369         Booking booking = new Booking(bookingId, customer, flight, seatNumber);
370         bookings.offer(booking);
371     }
372     reader.close();
373     return bookings;
374 }
375
376 // Main class
377 public class Vacation {
378     public static void main(String[] args) {
379         FlightManager manager = new FlightManager();
380
381         try {
382             // Load initial data from files
383             manager.addFlights(FileManager.loadFlights());
384             manager.addCustomers(FileManager.loadCustomers());
385             manager.addBookings(FileManager.loadBookings());
386         } catch (IOException e) {
387             System.err.println("Error loading data: " + e.getMessage());
388         }
389
390         // Sample operations
391         manager.addFlight("AI101", "DEL", "BOM");
392         manager.addFlight("AI102", "BOM", "DEL");
393
394         manager.addCustomer("C001", "Rahul");
395         manager.addCustomer("C002", "Priya");
396
397         manager.bookCustomer("B001", manager.getCustomer("C001"), manager.getFlight(
            "AI101"), "A1");
398         manager.bookCustomer("B002", manager.getCustomer("C002"), manager.getFlight(
            "AI102"), "B2");
399
400         // Save data to files
401         try {
402             FileManager.saveFlights(manager.getFlights());
403             FileManager.saveCustomers(manager.getCustomers());
404             FileManager.saveBookings(manager.getBookings());
405         } catch (IOException e) {
406             System.err.println("Error saving data: " + e.getMessage());

```

```
407         }
408
409         // Display flights, customers, and bookings
410         manager.listFlights();
411         manager.listCustomers();
412         manager.listBookings();
413     }
414 }
415
```